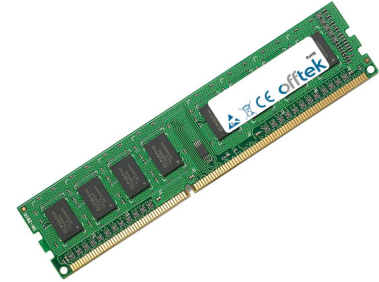


**Computer Engineering,
not Computer Science**



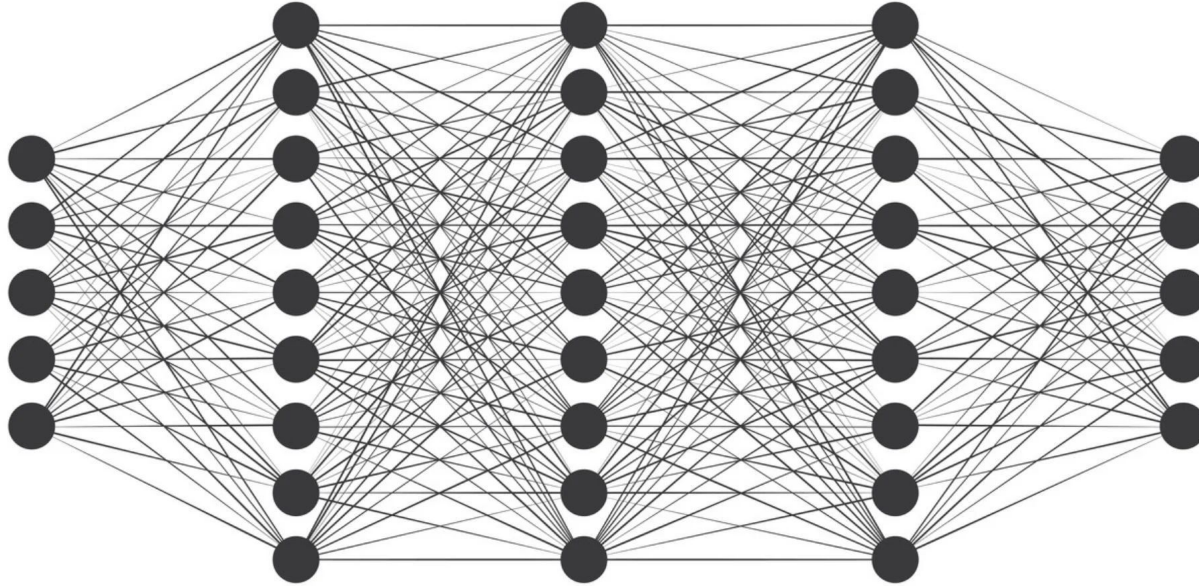


Computer Architecture

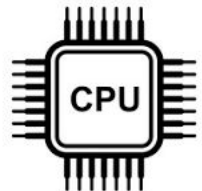
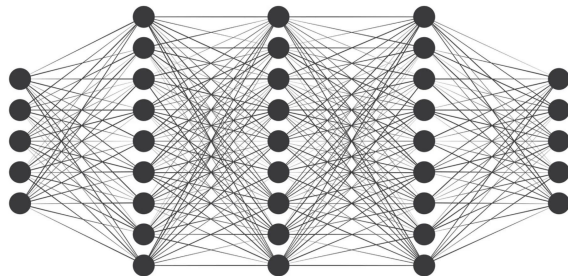


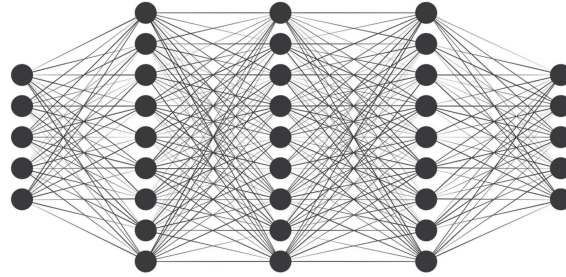


Artificial Neural Networks

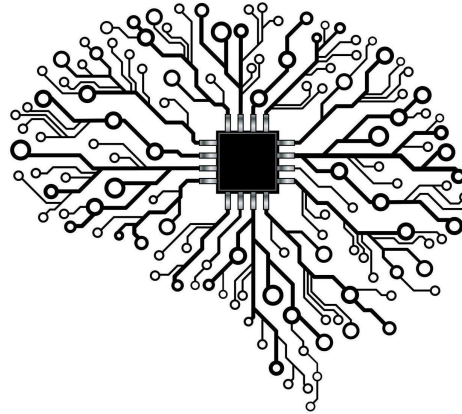


Deep Learning and Big Data brought the explosion of AI!





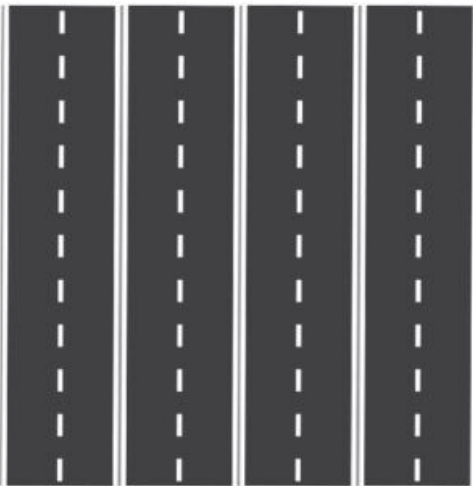
Neuromorphic Computing



- Brain Like
- Brings memory and compute elements closer together



CPU



GPU

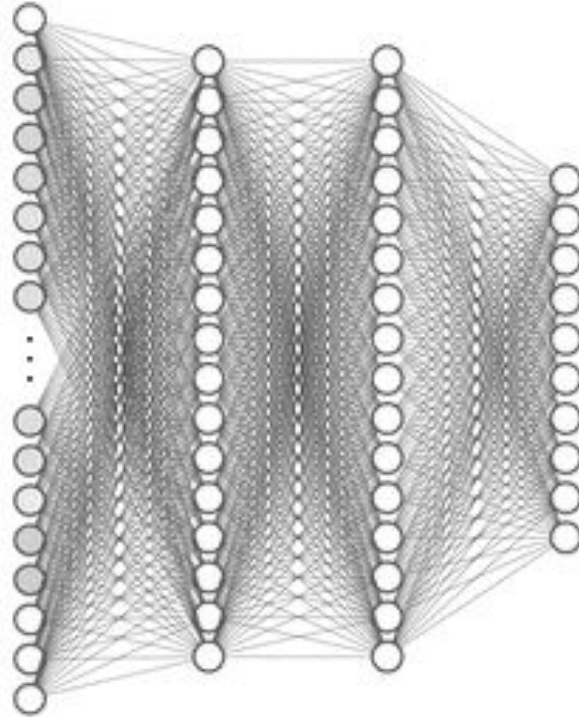


Neuromorphic Chip



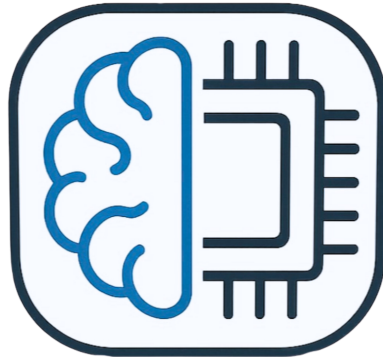
Spiking Neural Networks

- Sparsity = energy and computationally efficient



PeraMorphIQ

PeraCom Neuromorphic Research Group



Supervisors

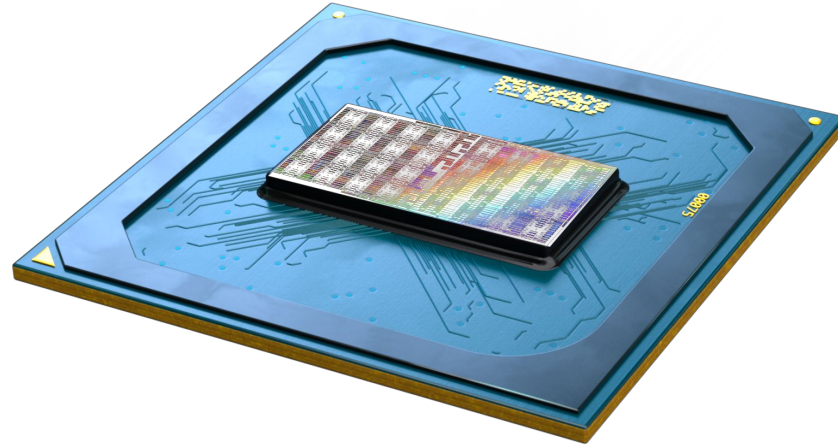


Dr. Isuru Nawinne

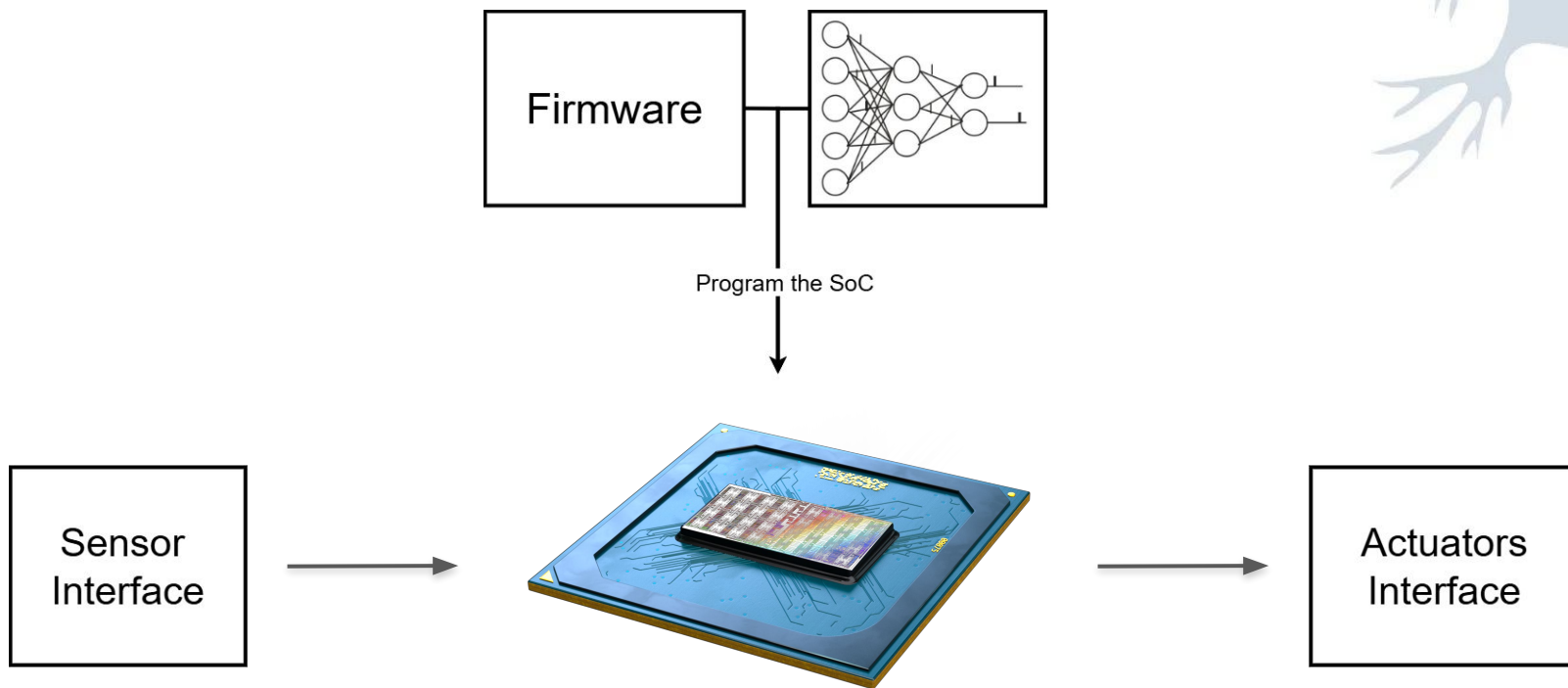


Prof. Roshan Ragel

What we are doing...



**SNAP-V: Spiking Neural Accelerated RISC-V
Processor**



**SNAP-
V**



Low Power Edge Systems with AI

Experience

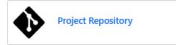
neuromorphic architecture

Created: 15-07-2022 Forks: 4 Watchers: 10 Stars: 10

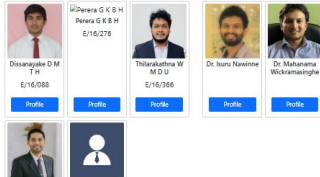


Description

Neuromorphic architectures are hardware architectures that use the biologically inspired neural functions as the basis of operation. Information processing based on spiking neuron architectures have caught considerable attention in recent years due to its low power consumption compared to traditional artificial neural networks. In this project, as the first stage, we are implementing parallel multiple processing elements based on RISC-V architecture to represent biological neurons. Single neurons can be implemented as a single processor with local memory access or since the spike time of biological neurons is in the millisecond order multiple neurons can be virtualized to a single processor. At the second stage of the process, we are expecting to design encoders and decoders to benchmark the architecture by solving classical machine learning problems.



Team / Supervisors



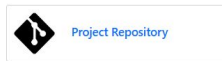
Neuromorphic NoC Architecture for SNNs

Created: 19-05-2023 Forks: 9 Watchers: 22 Stars: 22



Description

This project aims to develop a novel neuromorphic NoC architecture based on RISC-V ISA to support spiking neural network applications, and test it on FPGA.

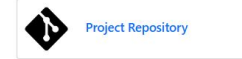


Team / Supervisors

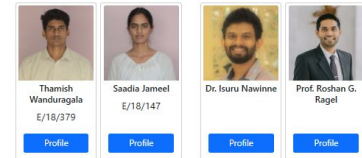


Neuromorphic NoC Architecture for SNNs

Created: 07-01-2024 Forks: 5 Watchers: 7 Stars: 7



Team / Supervisors



E19 - SNAP -V: A RISC -V SoC with Configurable Neuromorphic Acceleration for Small -Scale Spiking Neural Networks

Computer Science > Hardware Architecture

[Submitted on 12 Mar 2026]

SNAP-V: A RISC-V SoC with Configurable Neuromorphic Acceleration for Small-Scale Spiking Neural Networks

Kanishka Gunawardana, Sanka Peeris, Kavishka Rambukwella, Thamish Wanduragala, Saadia Jameel, Roshan Ragel, Isuru Nawinne

Spiking Neural Networks (SNNs) have gained significant attention in edge computing due to their low power consumption and computational efficiency. However, existing implementations either use conventional System on Chip (SoC) architectures that suffer from memory-processor bottlenecks, or large-scale neuromorphic hardware that is inefficient and wasteful for small-scale SNN applications. This work presents SNAP-V, a RISC-V-based neuromorphic SoC with two accelerator variants: Cerebra-S (bus-based) and Cerebra-H (Network-on-Chip (NoC)-based) which are optimized for small-scale SNN inference, integrating a RISC-V core for management tasks, with both accelerators featuring parallel processing nodes and distributed memory. Experimental results show close agreement between software and hardware inference, with an average accuracy deviation of 2.62% across multiple network configurations, and an average synaptic energy of 1.05 pJ per synaptic operation (SOP) in 45 nm CMOS technology. These results show that the proposed solution enables accurate, energy-efficient SNN inference suitable for real-time edge applications.

Comments: 12 pages, 5 figures, 5 tables

Subjects: **Hardware Architecture (cs.AR)**; Neural and Evolutionary Computing (cs.NE)

Cite as: [arXiv:2603.11939](https://arxiv.org/abs/2603.11939) [cs.AR]

(or [arXiv:2603.11939v1](https://arxiv.org/abs/2603.11939v1) [cs.AR] for this version)

<https://doi.org/10.48550/arXiv.2603.11939> 

Submission history

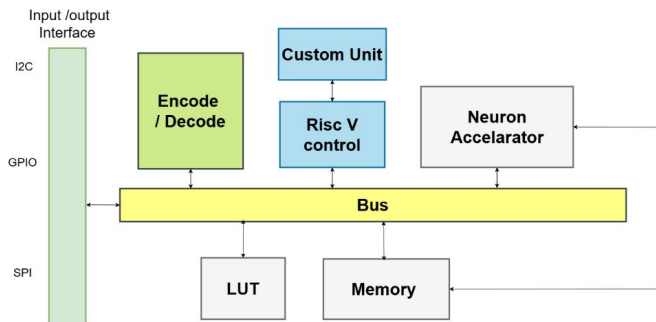
From: Kanishka Gunawardana [\[view email\]](#)

[v1] Thu, 12 Mar 2026 13:52:44 UTC (395 KB)

E20 - On-Chip Offline Neuromorphic Computing

Three-State On-Chip Pipeline

The full on-chip learning loop operates as three sequential states per training sample:



Team / Supervisors



S.M.P.H.
Samarakoon
E/20/346

[Profile](#)



Wakkumbura
M.M.S.S.
E/20/419

[Profile](#)



Wickramasinghe
J.M.W.G.R.L.
E/20/439

[Profile](#)



Dr. Isuru Nawinne



Prof. Roshan G.
Ragel

[Profile](#)

Methodology

Neuron Model

The system uses the **Leaky Integrate-and-Fire (LIF)** neuron model. At each discrete timestep:

```
V_mem[t] = β × V_mem[t-1] + Σ(w_i × spike_i[t])
if V_mem[t] ≥ V_threshold:
    spike_out = 1, V_mem = 0 (reset)
else:
    spike_out = 0
```

where $\beta = 192/256 \approx 0.75$ is the leak factor and all values are Q16.16 fixed-point.

Surrogate Gradient

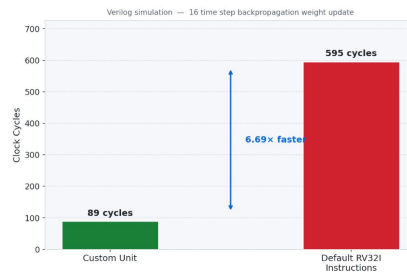
Because the Heaviside spiking function is non-differentiable, a **256-entry Q16.16 ROM LUT** stores pre-computed surrogate gradient values (smooth approximation of the sigmoid derivative). The LUT index is derived from the neuron's membrane potential V_{mem} .

Backpropagation

Weight updates use a surrogate-gradient backpropagation rule with momentum:

$$\delta = ((\text{error} + 0.95 \times \delta_{\text{prev}}) \times \text{surrogate_grad} \times \text{spike_status}) \gg 8$$
$$W' = W - (LR \times \delta) \gg 8 \quad (LR = 150/256 \approx 0.586)$$

This computation is accelerated by the custom RISC-V backprop unit, which includes two PISO LIFO buffers (one for spike status, one for gradients) and a DMA-style Memory-to-LIFO Loader FSM.



Neuromorphic Accelerator: Execution Phase Breakdown

E20 - On-Chip Online Learning For Neuromorphic

Experiment Setup and Implementation

To evaluate the partitioned EONS architecture, a comprehensive hardware-software co-simulation methodology was established. Verilator was utilized to perform cycle-accurate co-simulations. The core event-driven components were implemented at the Register-Transfer Level (RTL) and seamlessly interfaced with the software-designated evolutionary components written natively in C++.

The accelerator was benchmarked using the standard MNIST handwritten digit dataset using a rate-coding scheme. To calculate fitness efficiently, the C++ Inference Controller dynamically samples a representative subset (10%) of the total dataset for manual testing. Over a defined number of generations, the C++ Cross-Over unit generates new network parameters that are continually written back to the hardware Neuron Arrays.

Accelerator Design Diagram

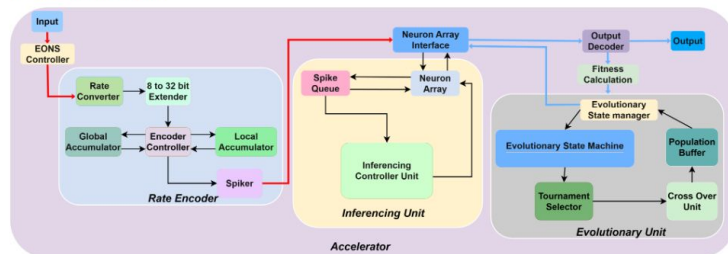
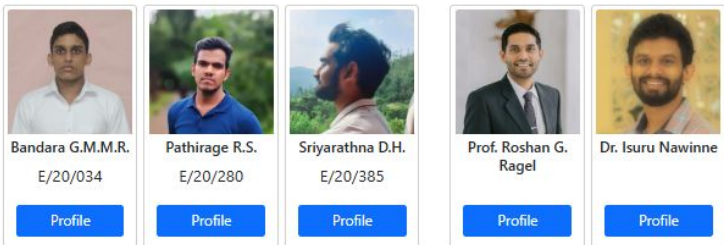


Figure 2: Detailed microarchitecture of the neuromorphic hardware accelerator.

Team / Supervisors



Methodology

The system operates using a Hardware-Software (HW/SW) co-design architecture to prevent overpopulating the silicon area with complex, non-critical control components. Computationally heavy genetic operations such as fitness calculation, tournament selection, and cross-over are partitioned into software.

Conversely, the high-throughput, event-driven data paths are implemented strictly in hardware. The hardware accelerator utilizes a Leaky Integrate-and-Fire (LIF) neuron model and consists of a Rate Encoder to convert real-valued inputs into spike trains, an Inference Unit (featuring a Network-on-Chip and banked neuron arrays), and an Evolutionary State Manager to handle the learning loops.

High-Level Design Diagram

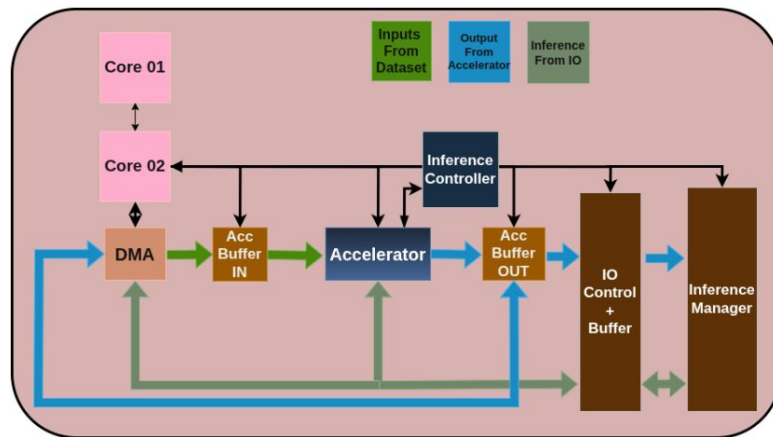
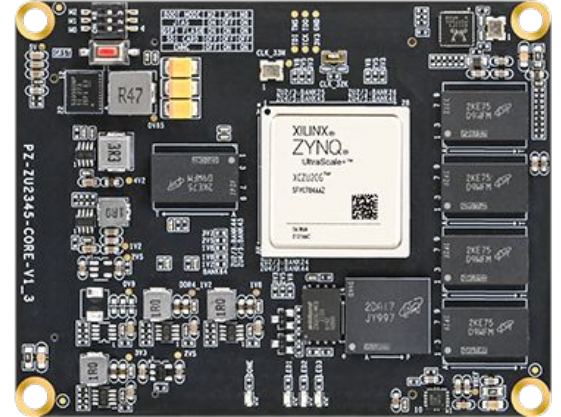


Figure 1: High-level architecture of the on-chip online learning neuromorphic accelerator.



E21 - Starting...

Servers and FPGAs



Copyright (c) 1988 - 2024 Synopsys, Inc.

This software and the associated documentation are proprietary to Synopsys, Inc. This software may only be used in accordance with the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, or distribution of this software is strictly prohibited. Licensed Products communicate with Synopsys servers for the purpose of providing software updates, detecting software piracy and verifying that customers are using Licensed Products in conformity with the applicable License Key for such Licensed Products. Synopsys will use information gathered in connection with this process to deliver software updates and pursue software pirates and infringers.

Inclusivity & Diversity - Visit SolvNetPlus to read the "Synopsys Statement on

Inclusivity and Diversity" (Refer to article 000036315 at

<https://solvnetplus.synopsys.com>)

set app_var power_enable_rtl_analysis true

Information: Checked out license 'PrimePower-RTL' (PT-019)



SYNOPSYS®

```
0[ 0.0%] 4[ 0.7%]
1[ 0.0%] 5[ 0.7%]
2[ 0.0%] 6[ 1.3%]
3[ 100.0%] 7[ 0.0%]
Mem[|||||] [51.6G/62.3G] Tasks: 130, 188 thr, 159 kthr; 2
Swp[|||||] [817M/274G] Load average: 1.03 1.22 1.53
Uptime: 21 days, 03:25:43
```

Main	I/O	PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	C
2010468	e19129	20	0	55.1G	52.4G	454M	R	99.9	82.2	11h43:00	/		
454	root	20	0	256M	3668	3608	S	0.7	0.0	4h01:59	@		
1131	root	20	0	605M	1068	276	S	0.7	0.0	34:46.74	/		
1402	root	20	0	605M	1068	276	S	0.7	0.0	17:41.58	/		
2094542	e19129	20	0	225M	4060	3252	R	0.7	0.0	0:00.06	h		
1	root	20	0	236M	7044	3820	S	0.0	0.0	2:46.90	/		
717	root	20	0	115M	8512	8060	S	0.0	0.0	0:05.31	/		
755	root	20	0	97M	1580	1280	S	0.0	0.0	0:02.78	/		
922	rpc	20	0	67328	328	304	S	0.0	0.0	0:00.79	/		
934	root	16	-4	127M	728	492	S	0.0	0.0	0:01.31	/		
935	root	16	-4	127M	728	492	S	0.0	0.0	0:00.01	/		
936	root	16	-4	48724	100	0	S	0.0	0.0	0:00.22	/		
937	root	16	-4	127M	728	492	S	0.0	0.0	0:00.16	/		
964	libstorage	20	0	8688	100	72	S	0.0	0.0	0:01.71	/		
966	polkitd	20	0	1970M	10112	6956	S	0.0	0.0	0:01.92	/		
968	root	20	0	532M	2064	500	S	0.0	0.0	4:39.59	/		
969	dbus	20	0	66124	1892	376	S	0.0	0.0	0:07.32	/		
970	rtkit	21	1	187M	284	256	S	0.0	0.0	0:07.30	/		
972	root	39	19	17752	36	0	S	0.0	0.0	0:00.13	/		
973	root	20	0	6756	332	332	S	0.0	0.0	0:00.00	/		
974	root	20	0	509M	1272	0	S	0.0	0.0	0:02.46	/		
975	root	20	0	83528	2344	1368	S	0.0	0.0	0:03.03	/		
976	root	20	0	122M	128	0	S	0.0	0.0	0:50.34	/		

```
echo "===== STEP 4: Report Power ====="
===== STEP 4: Report Power =====
# Group power consumption
report_power -group register > "results/power_register.txt"
report_power -group sequential > "results/power_sequential.txt"
report_power -group combinational > "results/power_combinational.txt"

report_power -group black_box > "results/power_black_box.txt"
report_power -group io_pad > "results/power_io_pad.txt"
report_power -hierarchy -levels 100 -verbose > "results/power_by_mod
ule_all.txt"
report_power -hierarchy -levels 20 -verbose > "results/power_by_modu
le_20.txt"
report_power -hierarchy -levels 10 -verbose > "results/power_by_modu
le_10.txt"
report_power -hierarchy -levels 5 -verbose > "results/power_by_modul
e_5.txt"
report_power -hierarchy -levels 3 -verbose > "results/power_by_modul
e_3.txt"
report_power -hierarchy -levels 2 -verbose > "results/power_by_modul
e_2.txt"
echo "===== STEP 5: RTL Metrics ====="
===== STEP 5: RTL Metrics =====
report_rtl_metrics -list > "results/rtl_metrics_list.txt"
report_rtl_metrics -view hier -hier_attributes {gated_registers icg_
cells latch_cells reg_cells sequential_cells combinational_cells tot
al_power} > "results/rtl_metrics_hier.txt"
report_rtl_metrics -view register -reg_attributes {dynamic_power swi
tching_power leakage_power total_power register_gated_root_clk_name}
> "results/rtl_metrics_register.txt"
echo "===== STEP 6: Check RTL Power ====="
===== STEP 6: Check RTL Power =====
check_rtl_power > "results/check_rtl_power.txt"
[===== 100%
```

F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice F8Nice F9K1
[blackbox:0:top*

"localhost.localdomain" 20:27 13-Jul-25

File Edit Flow Tools Reports Window Layout View Help Q- Quick Access

write_bitstream Complete ✓

Debug

Flow Navigator

- Create Block Design
- Open Block Design
- Generate Block Design
- SIMULATION
 - Run Simulation
- RTL ANALYSIS
 - Run Linter
 - Open Elaborated Design
- SYNTHESIS**
 - Run Synthesis
 - Open Synthesized Design
 - Constraints Wizard
 - Edit Timing Constraints
 - Set Up Debug
 - Open Dataflow Design
 - Report Timing Summary
 - Report Clock Networks
 - Report Clock Interaction

SYNTHESIZED DESIGN - xc7vx690tffg1761-2

Netlist

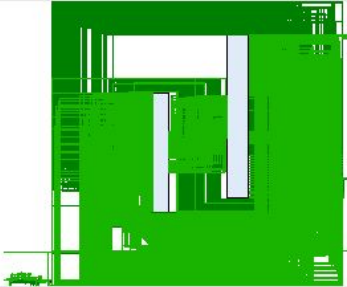
- cpu_fpga
 - Nets (3155)
 - Leaf Cells (65)
 - cpu_inst (cpu)
 - dbg_hub (dbg_hub_CV)

Properties

Select an object to see properties

Schematic

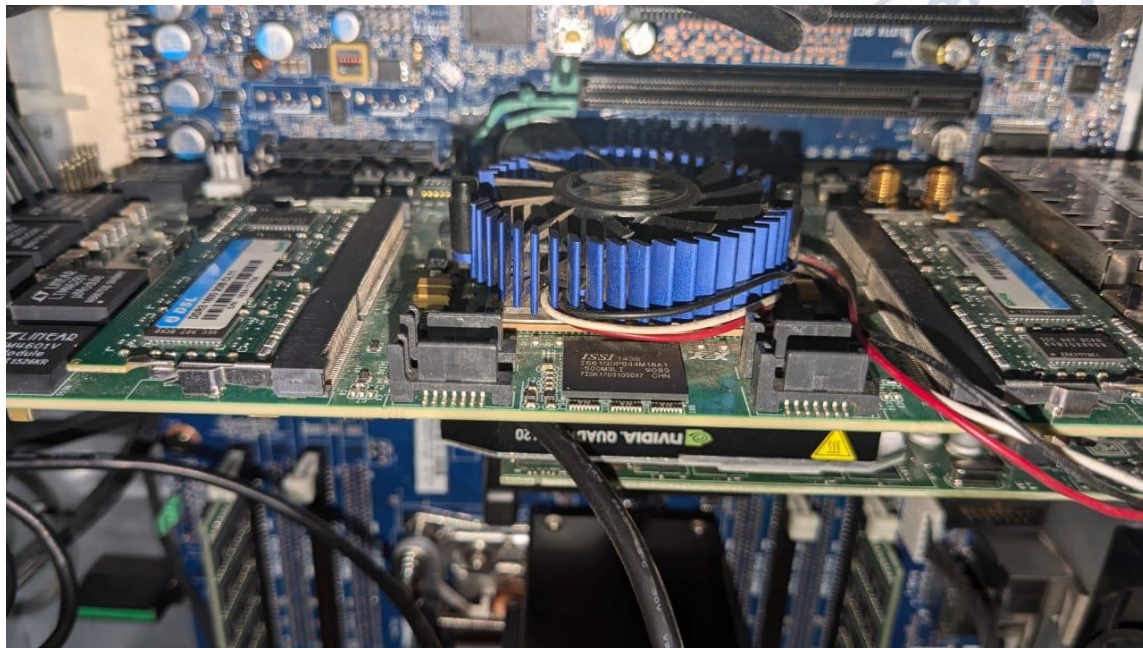
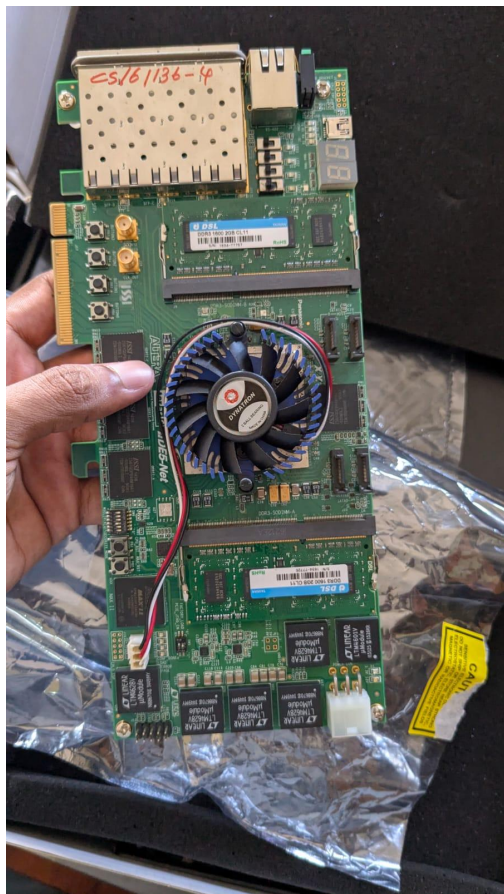
68 Cells 10 I/O Ports 3155 Nets



Tcl Console Messages Debug

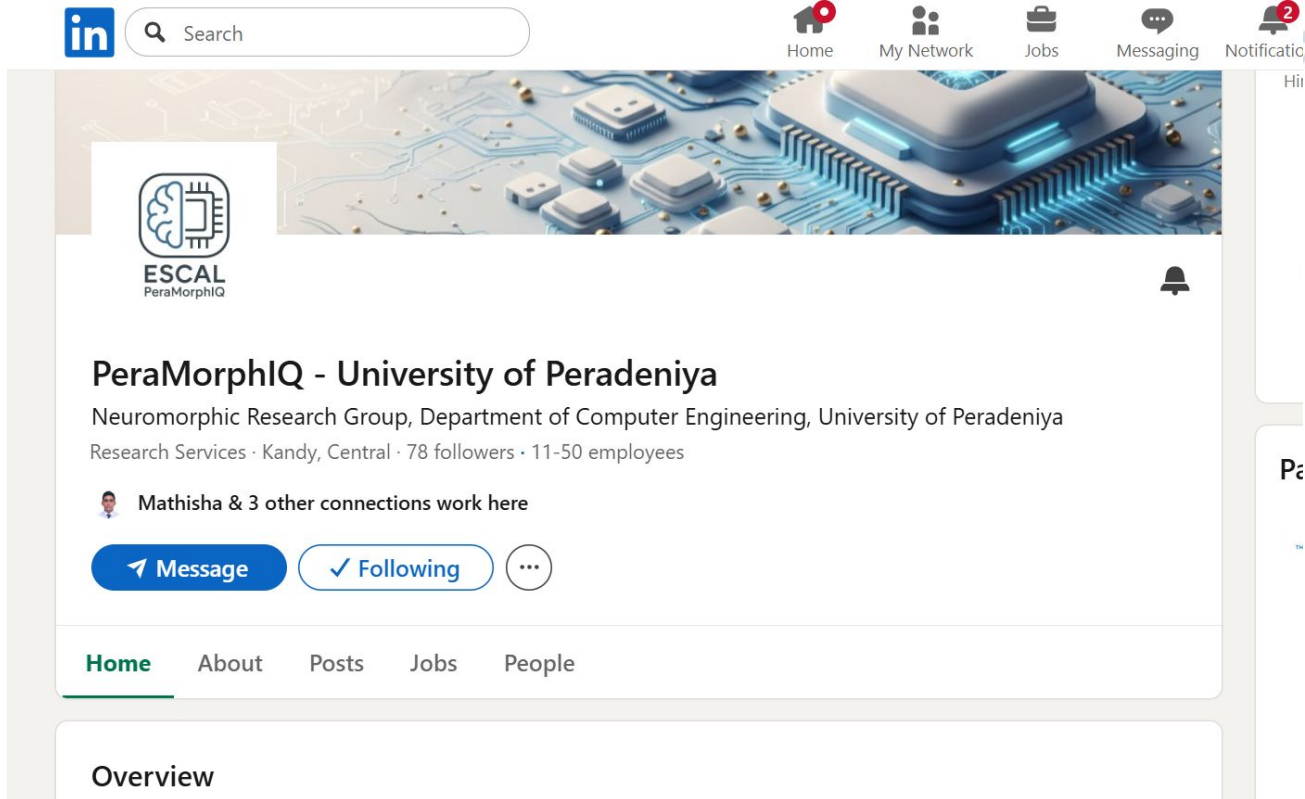
Name	Driver Cell	Driver Pin	Probe Type
dbg_hub(labtools_xsdcm_v3)			
u_ila_0(labtools_ila_v6)			
clk (1)			
probe0 (32)			Data and Trigger
probe1 (32)			Data and Trigger
probe2 (32)			Data and Trigger
probe3 (32)			Data and Trigger

Debug Cores Debug Nets





<https://www.linkedin.com/company/peramorphiq>



in Search

Home My Network Jobs Messaging Notifications

ESCAL
PeraMorphIQ

PeraMorphIQ - University of Peradeniya
Neuromorphic Research Group, Department of Computer Engineering, University of Peradeniya
Research Services · Kandy, Central · 78 followers · 11-50 employees

Mathisha & 3 other connections work here

Message Following

Home About Posts Jobs People

Overview

If you're interested in:

Building Processors

Working with Computer Hardware

Contributing to the NEXT BIG THING

Let us know - you're always welcome!!!

